LPP PREGUNTAS TEMA #05

1. **¿Qué es un bloque en Ruby?**

Un bloque es una porción de código encerrada entre llaves “{}” o entre “do...end” que puede pasarse como argumento.

2. **¿Qué se obtiene como salida? Describa el comportamiento del programa.**

```
class MyArray < Array
        def initialize (a_array)
                super (a_array)
        end
        def times_repeat (a_num)
                a_num.times do |num|
                        self.each do |i|
                                yield "[#{num}] :: ’#{i}’"
                        end
                …
end

a_array = MyArray.new([1,2,3])
a_array.times_repeat(2) do |x|
        puts x
end
```

Se imprimen las combinaciones del número de repetición y cada elemento del array:

```
[0] :: '1'
[0] :: '2'
[0] :: '3'
[1] :: '1'
[1] :: '2'
[1] :: '3'
```

El método “times_repeat” itera “a_num” veces y dentro de cada iteración recorre el array, pasando cada elemento al bloque con “yield”.

3. **¿Qué es un Proc?**

Es una clase que se utiliza para encapsular un bloque de código, convirtiéndolo en un objeto y permitiendo almacenarlo en una variable y ejecutarlo con “.call”.

---

5. **¿Qué es un lambda?**

Es una función que permite convertir un bloque de código en un objeto. A diferencia de Proc, sí comprueba los argumentos del bloque.

---

13. ¿Qué se entiende por efectos laterales?

Cambios en el estado del programa fuera del ámbito de una función, como modificar variables externas, hacer E/S, etc.

14. ¿Qué son las funciones de orden superior (high order functions)?

Son funciones que pueden recibir o devolver como argumento otra función.

15. ¿Qué es la memorización (memoization)?

Es una técnica de optimización que almacena los resultados de las llamadas a funciones para evitar recalcularlos.

---

27. ¿Qué es la reflexión o introspección?

Es la capacidad de un programa para examinar y modificar su estructura, comportamiento y propiedades en tiempo de ejecución.

28. ¿Qué es un contexto (binding)?

Es un objeto que encapsula el entorno de ejecución actual, permitiendo acceder a variables locales, constantes y métodos definidos en dicho contexto.

29. ¿Qué se entiende por metaprogramación?

Es la capacidad de un programa para escribir, modificar o manipular su propio código durante el tiempo de ejecución. En Ruby, esto se logra mediante técnicas como la creación dinámica de clases, definición de métodos en tiempo de ejecución, entre otras.

30. ¿Qué es un Lenguaje de Dominio Específico (DSL - Domain Specific Language)?

Es un mini-lenguaje diseñado para resolver problemas específicos de un dominio concreto de forma más expresiva y concisa que un lenguaje de propósito general.

31. ¿Qué diferencias hay entre “instance_eval”, “class_eval” y “eval”?

- **“eval”:** Evalúa una cadena como código Ruby en el contexto actual, sin restricciones específicas a instancias o clases.

- **“instance_eval”:** Evalúa un bloque en el contexto de una instancia de un objeto, permitiendo la modificación dinámica de su comportamiento.

- **“class_eval”:** Evalúa un bloque en el contexto de una clase, permitiendo la modificación dinámica de su estructura y métodos.

32. ¿Cuál es el peligro de usar `eval` en un entorno distribuido?

El principal riesgo es de seguridad, debido a la posibilidad de ejecutar código remoto no confiable. Esto puede llevar a vulnerabilidades de seguridad, como la ejecución de código malicioso.